

배터리 이상 징후 조기 탐지 AI 모델

2026.5.18 v1

Mode 1,2 – Input 가공

-> 실시간 data 축적 (100ms)

전압(Voltage)

- 범위 고정
- Raw 전압 데이터의 한계 -> 쇼트 때문에 특 떨어지는 이상 상태를 구별하기 어려움
- 변화량 도입 -> 전 100ms 전 전압과 지금 전압의 차이 (100ms= kafka에서 쌓은 Raw data시간 간격)
- `df['V_delta'] = df['voltage'].diff(periods=1)`

```
# 순수 수학 공식대로 0 ~ 1 공간으로 압축
```

```
df['V_scaled'] = (df['voltage'] - V_min) / (V_max - V_min)
```

```
# 직전 행(Row)과의 차이를 구함 (기본값 periods=1)
```

```
df['V_delta'] = df['voltage'].diff(periods=1)
```

전류(Current)

- 하드웨어 특성상 노이즈가 많이 끼고, 범위가 고정되어 있지 않음
- Step1 : 이동 평균 필터 (Moving Average Filter)
-> `rolling(window=5).mean()`을 사용 (100ms 기준으로 최근 0.5초(5개 스텝) 동안의 전류 평균값)
- Step2 : Standard Scaling (Z-score 표준화)
- `df['I_smooth'] = df['current'].rolling(window=5, min_periods=1).mean()`

```
# 최근 5개 행(0.5초)의 평균을 구하되, 처음 1~4개 행에 NaN이 생기는 것을 방지
df['I_smooth'] = df['current'].rolling(window=5, min_periods=1).mean()
```

```
# 평균( $\mu$ )과 표준편차( $\sigma$ ) 산출
mean_I = df['I_smooth'].mean()
std_I = df['I_smooth'].std()

# Z-score 수식 그대로 대입
df['I_scaled'] = (df['I_smooth'] - mean_I) / std_I
```

온도(Temperature)

- 보통 기존 BMS(배터리 관리 시스템)들은 Threshold 방식-> 이 방식은 열폭주를 '예측'할 수 없음
-> 열 지연 현상(Time-Lag) 때문 -> 열이 센서까지 전달되어 절대적인 온도 숫자를 나타낼 때 까지의 시간
- 1차, 2차 미분 -> 온도의 변화량, 가속도 도입
- 온도, 변화량, 가속도로 종합 판단

```
# 1단계: 온도 1차 차분 (100ms 동안의 온도 변화량 = 속도)
df['dT_dt'] = df['temperature'].diff(periods=1).fillna(0)

# 2단계: 온도 2차 차분 (온도 변화량의 변화량 = 가속도)
df['d2T_dt2'] = df['dT_dt'].diff(periods=1).fillna(0)
```

SOC(State of Charge, 충전 상태)

- 리튬이온 배터리는 완전히 찼을 때(4.2V)와 완전히 빌 때(3.0V)를 제외하고, 중간 구간(SOC 20% ~ 80%)에서는 전압이 3.7V~3.8V 근처에서 거의 변하지 않고 평평하게 유지되는 성질 존재
-> 전압 숫자만 보서는 정확한 잔여 에너지를 알 수 없음->bq27441 칩이 알아서 SOC 계산
- 정상 상태에서는 전류가 흐르는 만큼 SOC 와 전압이 동기화되어 같이 움직임. 하지만 내부 단락(Short)이 생기면 이 동기화(Coupling)가 완전히 붕괴 ->?
- SOC 자체가 % 지표이기 때문에 전처리는 SOC/100만 진행

```
# 0~100 범위의 SOC를 딥러닝이 좋아하는 0~1 범위로 쪼그라뜨림  
df['SOC_scaled'] = df['soc'] / 100.0
```

SOH (State of Health, 배터리 노화도)

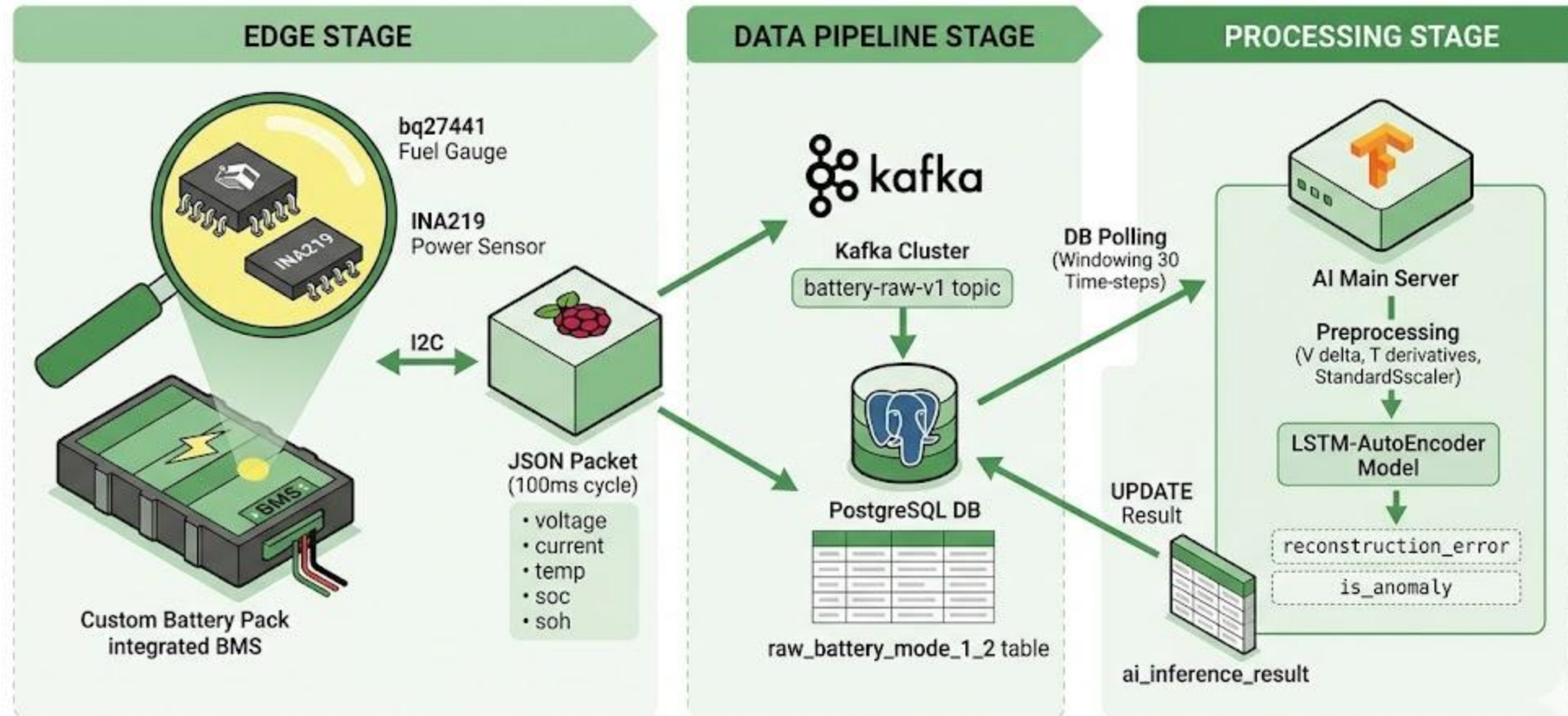
- AI는 노후 배터리가 정상적으로 일할 때 발생하는 큰 전압 강하와 빠른 온도 상승을 보고 SOC가 없다면 열 폭주상태로 오판단 가능하기 때문에 SOC가 맥락 변수(Context Feature)역할
- SOH와 마찬가지로 %지표

```
# 0~100 범위의 SOH를 딥러닝 매트릭스 체급에 맞게 0~1로 압축  
df['SOH_scaled'] = df['soh'] / 100.0
```

Mode 1,2

VERSION 1: INTERNAL BATTERY PACK LINE (MODE 1 & 2)

PROJECT DREAM UP: BATTERY THERMAL RUNAWAY DETECTION



Mode 3 – Input 가공

-> 계속 축적 X , 특정 시점에 data 저장, 평소에는 data 0으로 저장

순간 전압 강하량

- 내부 단락 불량이나 진행 중이거나 심하게 노화된 배터리는 저항이 비정상적으로 커져 있기 때문에, 동일한 전류 부하를 걸어도 강하량 높음
- 측정 전 전압 data(100ms 전 전압)와 현재 전압의 차이의 절댓값 -> Z-score 표준화

```
# 1. 100ms 전 전압과 현재 전압의 차이를 구함
df['V_diff'] = df['ext_voltage'].diff(periods=1)

# 2. 부하가 걸릴 때 '음수'로 특 떨어지므로, abs()를 취해 전압 강하량(양수 변별력)으로 변환
df['V_drop'] = df['V_diff'].abs()

# 3. 결측치 제거 및 데이터 체급 조절을 위한 StandardScaler 가공 예 대기
```

온도 상승 기울기 + 칼만필터

- 외부 시판 보조배터리의 껍데기를 뜯지 않고 단속할 때 발생하는 물리적인 한계(열 지연과 외부 노이즈)를 AI 서버단에서 소프트웨어(수학)로 완벽하게 극복-> 열 지연현상 + 환경 노이즈
- 메인 서버 파이썬 스크립트에서 AI 구동 -> 칼만필터 라이브러리 사용 가능 -> 필터링 된 온도 data 사용 = 모드 1,2 온도 input과 동일(온도, 변화량, 가속도)

```
class KalmanFilter1D:
    def __init__(self, Q, R):
        self.Q = Q # 프로세스 노이즈 공분산 (시스템 자체의 신뢰도)
        self.R = R # 측정 노이즈 공분산 (센서가 흔들리는 정도)
        self.p = 1.0 # 오차 공분산 초기화
        self.x = None # 추정된 진짜 온도 상태

    def update(self, measurement):
        if self.x is None:
            self.x = measurement
            return self.x

        # 1) 예측 (Prediction)
        # 2) 칼만 이득 계산 (Kalman Gain)
        k_gain = self.p / (self.p + self.R)
        # 3) 상태 업데이트 (Correction)
        self.x = self.x + k_gain * (measurement - self.x)
        # 4) 오차 공분산 업데이트
        self.p = (1 - k_gain) * self.p + self.Q
        return self.x

# AI 서버단 전처리 파이프라인 함수
def preprocess_mode3_temperature(df, Q=0.01, R=0.5):
    kf = KalmanFilter1D(Q, R)

    # 1단계 : 날것의 표면 온도에 실시간 칼만 필터 적용 (노이즈 완벽 평활화)
    df['T_kf'] = [kf.update(t) for t in df['raw_surface_temp']]

    # 2단계 : 필터링된 온도를 기반으로 100ms 주방의 상승 기울기(차분) 산출
    df['dT_kf_dt'] = df['T_kf'].diff(periods=1).fillna(0)

    return df
```

Issue 1 – 칼만 필터 및 input 가공연산 위치(라즈베리파이 VS 서버)

- 칼만필터 및 가공연산은 연산량이 꽤 많은 작업

가공연산 위치→	라즈베리파이	메인 서버	우리 프로젝트의 선택 가이드
연산 속도/파워	✗ 낮음 (LSTM+칼만 돌리기 버거움)	○ 매우 높음 (여유로운 고속 연산)	서버 승 (복잡한 AI 연산 필수)
MLOps / 유지보수	✗ 지옥 (기기마다 찾아가서 업데이트)	○ 천국 (서버 코드 한 줄 바꾸면 일괄 적용)	서버 승 (파라미터 튜닝 수시로 발생)
네트워크 끊겼을 때	○ 안전 (자체 판단 후 릴레이 차단 가능)	✗ 위험 (서버 통신 죽으면 진단 마비)	엣지 승 (하드웨어 안전장치 필요)
데이터 활용성	✗ 낮음 (정제된 데이터만 남아 재학습 불가)	○ 높음 (Raw 데이터 누적으로 모델 고도화)	서버 승 (학습 데이터 축적 필수)

Issue 2 - 칼만필터 튜닝용 초기 실험 설계

- 칼만필터 적용한 온도 VS 실제 배터리 내부 온도를 비교해서 칼만 필터의 정확도를 측정
- 칼만필터 최적의 하이퍼파라미터 찾기

-> Q(프로세스 노이즈 공분산): 배터리 온도가 열역학 법칙에 따라 매끄럽게 변할 것이라는 **우리 예측 시스템에 대한 불확실성**

-> R(측정 노이즈 공분산)에어컨 바람, 미세 흔들림 등으로 인해 **적외선 센서가 뱉는 숫자의 불확실성 (노이즈)**

1

-> Best Q, R 을 찾는 과정이 필요

전력량

- 매 스텝마다 전압(V) × 전류(A)로 현재 전력(W)을 구한 뒤, 수집 주기인 **0.1초**를 곱해 에너지 계산
- 정상 배터리-> 함수가 이 소량의 에너지를 100ms마다 누적(Cumulative Sum), Wh_cumsum 그래프는 방전이 진행될수록 완만하게 **우상향하는 매끄러운 곡선**
- 외부로 빠져나간 전력량 매트릭스 추이와 전압이 내려가는 곡선의 싱크 확인

```
# 1단계: 실시간 전력(Power, Watt 단위) 계산 (방전 전류이므로 abs 취해 양수 변환)
df['Power_W'] = df['ext_voltage'] * (df['ext_current'].abs() / 1000.0)

# 2단계: 100ms 주방(0.1초) 동안 소모된 에너지를 시간(Hour) 단위로 환산
# 0.1초 = 0.1 / 3600 시간
df['Energy_per_step'] = df['Power_W'] * (0.1 / 3600.0)

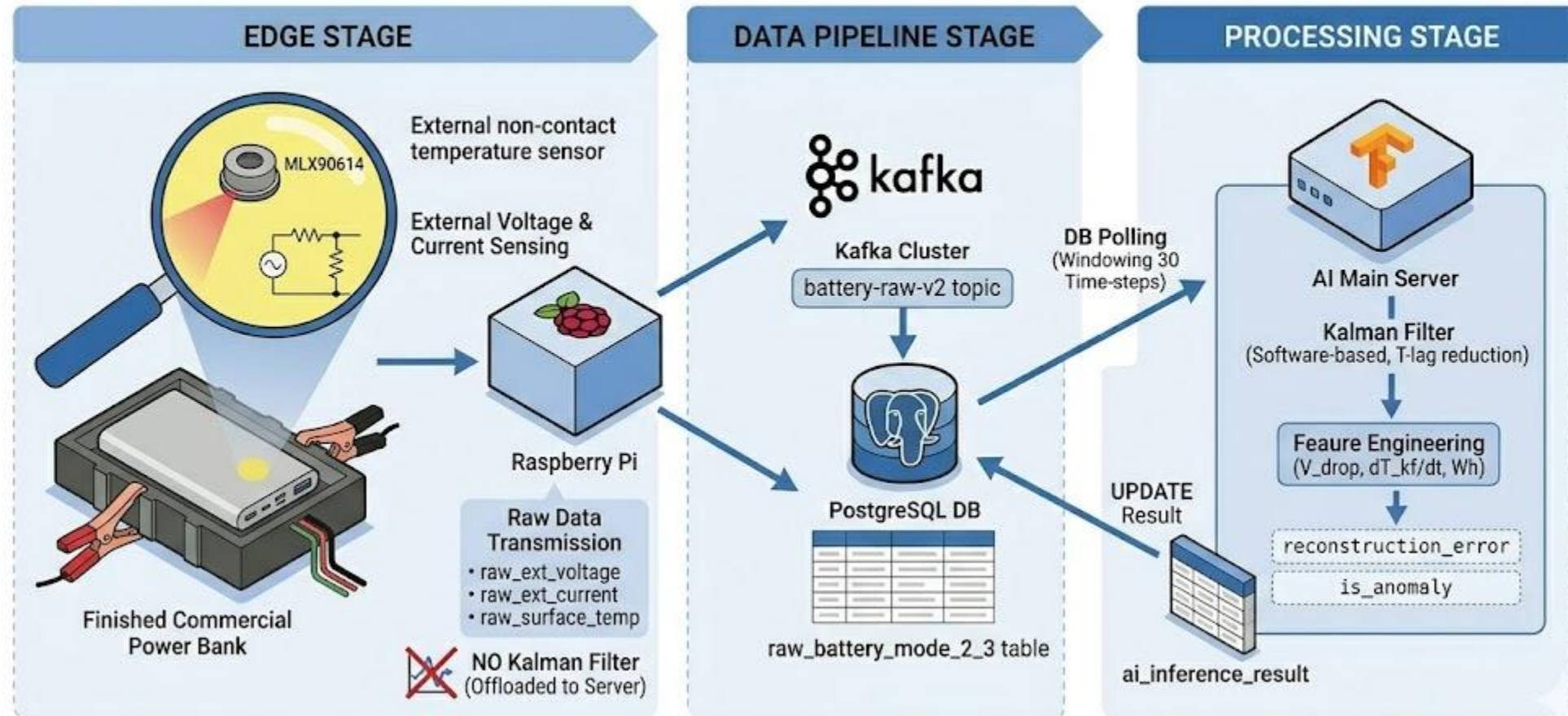
# 3단계: 누적합(Cumulative Sum)을 통해 최종 Wh 피쳐 생성
df['Wh_cumsum'] = df['Energy_per_step'].cumsum()

# 4단계: 체급 조절을 위한 Standard Scaling 적용 대기
```

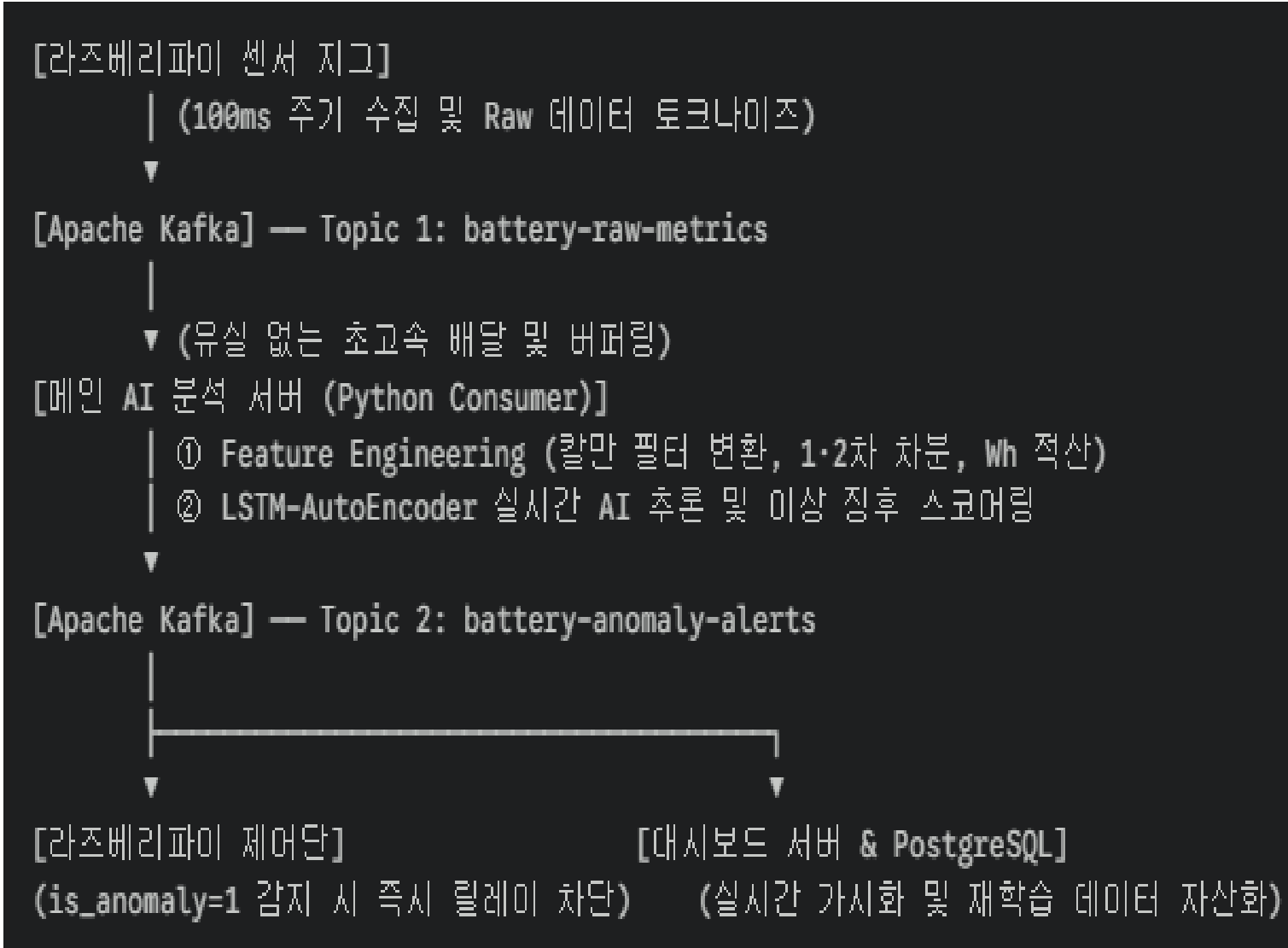
Mode 3

VERSION 2: EXTERNAL FINISHED BATTERY PACK LINE (MODE 3)

PROJECT DREAM UP: BATTERY THERMAL RUNAWAY DETECTION



Full
Architecture



Issue

Issue 3 – 이상데이터 확보 방식

Memo

- 추가 input은 버전1 돌려보고 평가지표 확인 후 판단
- Data가 필요
- 지원금 사용